Experiment No. 3

Ridge and Lasso Regression on the Diabetes Dataset: Implementation and Comparison with Standard Linear Regression

Name: MRIDUL SANKAR KV
Roll Number: 44
Batch: BATCH 2
Date: 06 August 2026

PROGRAM & OUTPUT

1. Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

sns.set_style("whitegrid")
RANDOM_STATE = 42
```

2. Load and Explore the Dataset

```
data = load_diabetes(as_frame=True)
df = data.frame
X = data.data
y = data.target

print("Dataset shape:", df.shape)
df.head()
```

```
Dataset shape: (442, 11)
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 11 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   age     442 non-null    float64
 1   sex     442 non-null    float64
 2   bmi     442 non-null    float64
 3   bp      442 non-null    float64
 4   s1      442 non-null    float64
 5   s2      442 non-null    float64
 6   s3      442 non-null    float64
 7   s4      442 non-null    float64
 8   s5      442 non-null    float64
 9   s6      442 non-null    float64
 10  target  442 non-null    float64
dtypes: float64(11)
```

```
print("Missing values per column:")
print(df.isnull().sum())
print("\nTotal missing values:", df.isnull().sum().sum())
```

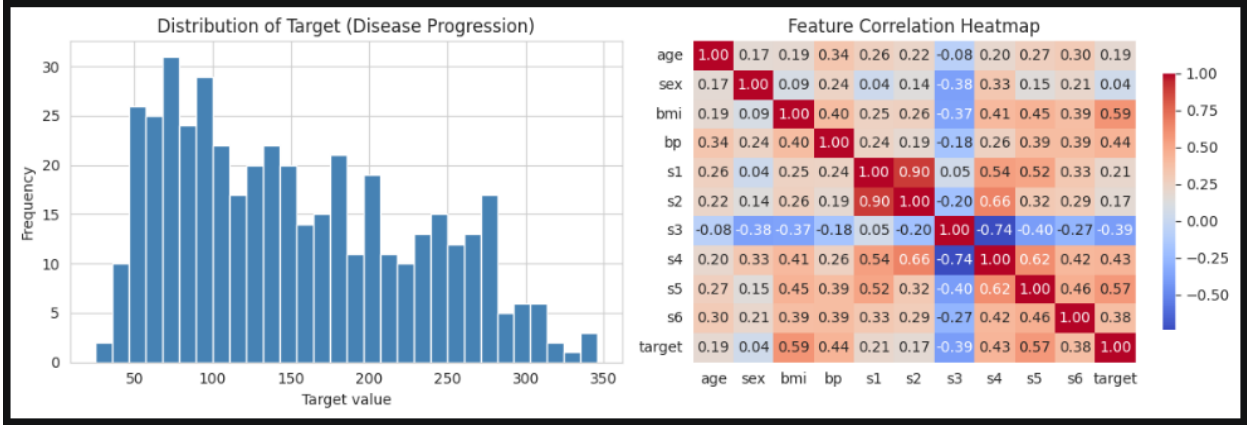
```
Missing values per column:
age       0
sex       0
bmi       0
bp        0
s1        0
s2        0
s3        0
s4        0
s5        0
s6        0
target    0
dtype: int64

Total missing values: 0
```

```
df.describe()
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
count	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	442.000000
mean	-2.511817e-19	1.230790e-17	-2.245564e-16	-4.797570e-17	-1.381499e-17	3.918434e-17	-5.777179e-18	-9.042540e-18	9.268604e-17	1.130318e-17	152.133484
std	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	77.093005
min	-1.072256e-01	-4.464164e-02	-9.027530e-02	-1.123988e-01	-1.267807e-01	-1.156131e-01	-1.023071e-01	-7.639450e-02	-1.260971e-01	-1.377672e-01	25.000000
25%	-3.729927e-02	-4.464164e-02	-3.422907e-02	-3.665608e-02	-3.424784e-02	-3.035840e-02	-3.511716e-02	-3.949338e-02	-3.324559e-02	-3.317903e-02	87.000000
50%	5.383060e-03	-4.464164e-02	-7.283766e-03	-5.670422e-03	-4.320866e-03	-3.819065e-03	-6.584468e-03	-2.592262e-03	-1.947171e-03	-1.077698e-03	140.500000
75%	3.807591e-02	5.068012e-02	3.124802e-02	3.564379e-02	2.835801e-02	2.984439e-02	2.931150e-02	3.430886e-02	3.243232e-02	2.791705e-02	211.500000
max	1.107267e-01	5.068012e-02	1.705552e-01	1.320436e-01	1.539137e-01	1.987880e-01	1.811791e-01	1.852344e-01	1.335973e-01	1.356118e-01	346.000000

Target variable distribution

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(y, bins=30, color='steelblue', edgecolor='white')
axes[0].set_title('Distribution of Target (Disease Progression)')
axes[0].set_xlabel('Target value')
axes[0].set_ylabel('Frequency')

corr = df.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', ax=axes[1], cbar_kws={'shrink': 0.8})
axes[1].set_title('Feature Correlation Heatmap')
plt.tight_layout()
plt.savefig('eda_plots.png', dpi=120)
plt.show()
```



3. Preprocessing: Train-Test Split and Feature Scaling

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training set size:", X_train_scaled.shape)
print("Test set size:", X_test_scaled.shape)
```

```
Training set size: (353, 10)
Test set size: (89, 10)
```

4. Standard Linear Regression (Baseline)

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)
y_pred_lin = lin_reg.predict(X_test_scaled)

mse_lin = mean_squared_error(y_test, y_pred_lin)
rmse_lin = np.sqrt(mse_lin)
mae_lin = mean_absolute_error(y_test, y_pred_lin)
r2_lin = r2_score(y_test, y_pred_lin)

print(f"Linear Regression -> MSE: {mse_lin:.3f}, RMSE: {rmse_lin:.3f}, MAE: {mae_lin:.3f}, R2: {r2_lin:.4f}")
```

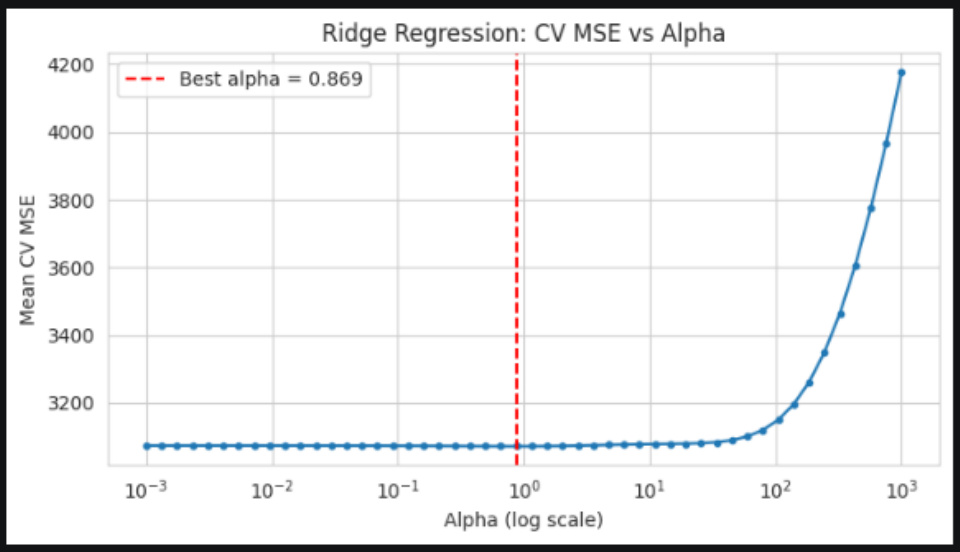
```
inear Regression -> MSE: 2900.194, RMSE: 53.853, MAE: 42.794, R2: 0.4526
```

5. Ridge Regression with Cross-Validated Hyperparameter Tuning

```
ridge_params = {'alpha': np.logspace(-3, 3, 50)}
kf = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

ridge_grid = GridSearchCV(
    Ridge(random_state=RANDOM_STATE),
    ridge_params,
    cv=kf,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)
ridge_grid.fit(X_train_scaled, y_train)

best_ridge = ridge_grid.best_estimator_
print("Best Ridge alpha:", ridge_grid.best_params_['alpha'])

y_pred_ridge = best_ridge.predict(X_test_scaled)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
rmse_ridge = np.sqrt(mse_ridge)
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print(f"Ridge Regression -> MSE: {mse_ridge:.3f}, RMSE: {rmse_ridge:.3f}, MAE: {mae_ridge:.3f}, R2: {r2_ridge:.4f}")
```

```
Best Ridge alpha: 0.868511373751352
Ridge Regression -> MSE: 2892.783, RMSE: 53.785, MAE: 42.810, R2: 0.4540
```

```python
# Visualize how CV MSE changes with alpha for Ridge
ridge_results = pd.DataFrame(ridge_grid.cv_results_)
plt.figure(figsize=(7, 4))
plt.plot(ridge_results['param_alpha'], -ridge_results['mean_test_score'], marker='o', ms=3)
plt.axvline(ridge_grid.best_params_['alpha'], color='red', linestyle='--',
            label=f"Best alpha = {ridge_grid.best_params_['alpha']:.3f}")
plt.xscale('log')
plt.xlabel('Alpha (log scale)')
plt.ylabel('Mean CV MSE')
plt.title('Ridge Regression: CV MSE vs Alpha')
plt.legend()
plt.tight_layout()
plt.savefig('ridge_cv_curve.png', dpi=120)
plt.show()
```

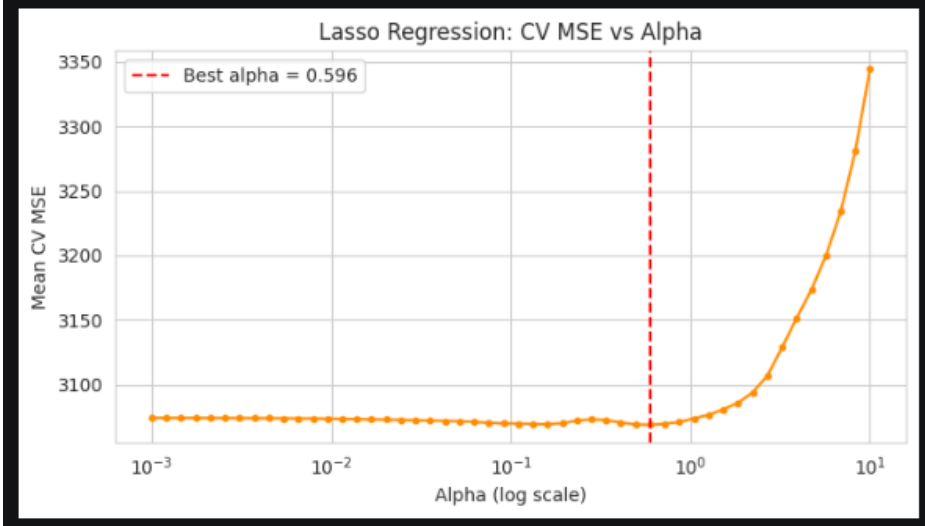


6. Lasso Regression with Cross-Validated Hyperparameter Tuning

```python
lasso_params = {'alpha': np.logspace(-3, 1, 50)}

lasso_grid = GridSearchCV(
    Lasso(random_state=RANDOM_STATE, max_iter=10000),
    lasso_params,
    cv=kf,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)
lasso_grid.fit(X_train_scaled, y_train)

best_lasso = lasso_grid.best_estimator_
print("Best Lasso alpha:", lasso_grid.best_params_['alpha'])

y_pred_lasso = best_lasso.predict(X_test_scaled)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
rmse_lasso = np.sqrt(mse_lasso)
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print(f"Lasso Regression -> MSE: {mse_lasso:.3f}, RMSE: {rmse_lasso:.3f}, MAE: {mae_lasso:.3f}, R2: {r2_lasso:.4f}")
```

```
Best Lasso alpha: 0.5963623316594643
Lasso Regression -> MSE: 2850.392, RMSE: 53.389, MAE: 42.832, R2: 0.4620
```

```python
lasso_results = pd.DataFrame(lasso_grid.cv_results_)
plt.figure(figsize=(7, 4))
plt.plot(lasso_results['param_alpha'], -lasso_results['mean_test_score'], marker='o', ms=3, color='darkorange')
plt.axvline(lasso_grid.best_params_['alpha'], color='red', linestyle='--',
            label=f"Best alpha = {lasso_grid.best_params_['alpha']:.3f}")
plt.xscale('log')
plt.xlabel('Alpha (log scale)')
plt.ylabel('Mean CV MSE')
plt.title('Lasso Regression: CV MSE vs Alpha')
plt.legend()
plt.tight_layout()
plt.savefig('lasso_cv_curve.png', dpi=120)
plt.show()
```

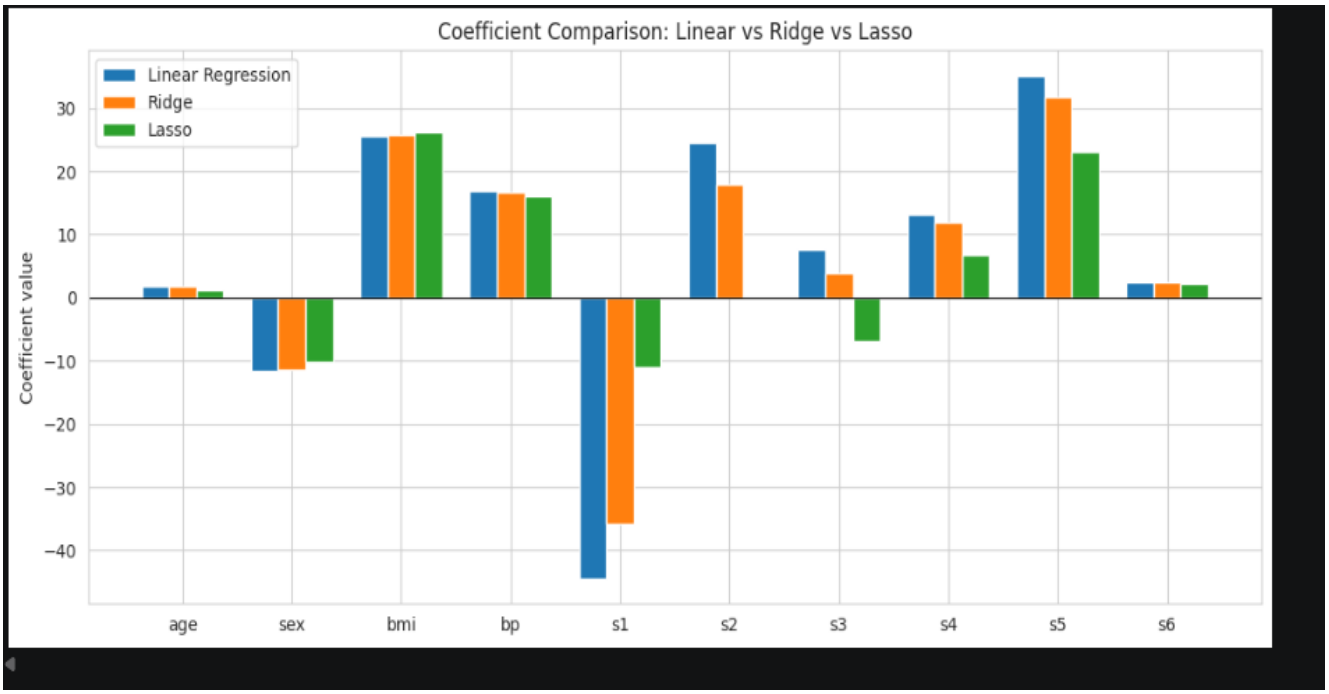


7. Coefficient Comparison

```python
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Linear Regression': lin_reg.coef_,
    'Ridge': best_ridge.coef_,
    'Lasso': best_lasso.coef_
})
coef_df
```

	Feature	Linear Regression	Ridge	Lasso
0	age	1.753758	1.801763	1.215956
1	sex	-11.511809	-11.455970	-10.180045
2	bmi	25.607121	25.721360	26.224828
3	bp	16.828872	16.745047	16.070299
4	s1	-44.448856	-35.687467	-11.001360
5	s2	24.640954	17.839644	-0.000000
6	s3	7.676978	3.817045	-6.931610
7	s4	13.138784	11.907778	6.677361
8	s5	35.161195	31.772174	22.998411
9	s6	2.351364	2.446083	2.285060

```python
x = np.arange(len(X.columns))
width = 0.25
plt.figure(figsize=(11, 5))
plt.bar(x - width, coef_df['Linear Regression'], width, label='Linear Regression')
plt.bar(x, coef_df['Ridge'], width, label='Ridge')
plt.bar(x + width, coef_df['Lasso'], width, label='Lasso')
plt.axhline(0, color='black', linewidth=0.8)
plt.xticks(x, X.columns)
plt.ylabel('Coefficient value')
plt.title('Coefficient Comparison: Linear vs Ridge vs Lasso')
plt.legend()
plt.tight_layout()
plt.savefig('coef_comparison.png', dpi=120)
plt.show()
n_zero = np.sum(np.abs(best_lasso.coef_) < 1e-6)
print(f"Number of features zeroed out by Lasso: {n_zero} ({X.columns[np.abs(best_lasso.coef_) < 1e-6].tolist()})")
```

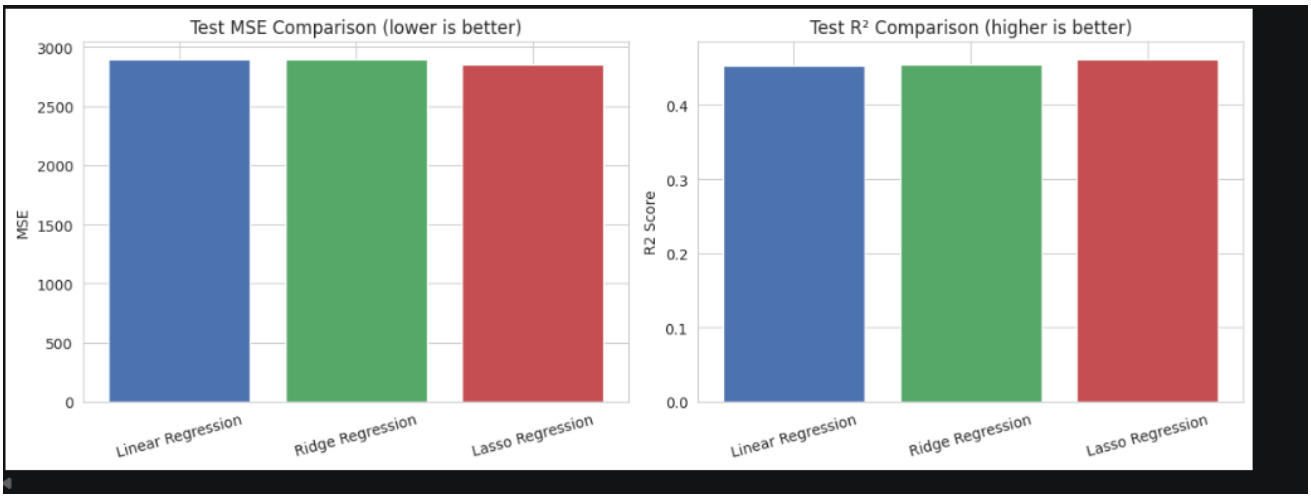


8. Performance Comparison: MSE and R^2

```python
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Ridge Regression', 'Lasso Regression'],
    'Best Alpha': ['-', round(ridge_grid.best_params_['alpha'], 4), round(lasso_grid.best_params_['alpha'], 4)],
    'MSE': [mse_lin, mse_ridge, mse_lasso],
    'RMSE': [rmse_lin, rmse_ridge, rmse_lasso],
    'MAE': [mae_lin, mae_ridge, mae_lasso],
    'R2 Score': [r2_lin, r2_ridge, r2_lasso]
})
results
```

	Model	Best Alpha	MSE	RMSE	MAE	R2 Score
0	Linear Regression	-	2900.193628	53.853446	42.794095	0.452603
1	Ridge Regression	0.8685	2892.783418	53.784602	42.810177	0.454001
2	Lasso Regression	0.5964	2850.392085	53.389063	42.832453	0.462003

```python
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

colors = ['#4C72B0', '#55A868', '#C44E52']
axes[0].bar(results['Model'], results['MSE'], color=colors)
axes[0].set_title('Test MSE Comparison (lower is better)')
axes[0].set_ylabel('MSE')
axes[0].tick_params(axis='x', rotation=15)

axes[1].bar(results['Model'], results['R2 Score'], color=colors)
axes[1].set_title('Test R² Comparison (higher is better)')
axes[1].set_ylabel('R2 Score')
axes[1].tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.savefig('performance_comparison.png', dpi=120)
plt.show()
```



REPORT:

EXPERIMENT NO :3

Ridge and Lasso regression

Aim :

To implement Ridge and Lasso regression on the Diabetes dataset and compare their performance against standard Ordinary Least Squares (OLS) Linear Regression using cross-validated hyperparameter tuning and standard regression metrics (MSE, RMSE, MAE, R^2).

Algorithm

1. **Load Data & Preprocessing:** Load the Diabetes dataset consisting of 442 patient records and 10 baseline physiological features. Check for missing values and summarize statistical metrics.
2. **Data Splitting & Feature Scaling:** Split the features (X) and target variable (y) into training and testing sets. Normalize/scale the baseline features using StandardScaler.
3. **Model Training & Hyperparameter Tuning:**
    - Fit a standard **Linear Regression (OLS)** baseline model.
    - Tune penalty parameters (α) for **Ridge ($\mathbf{L_2}$)** and **Lasso ($\mathbf{L_1}$)** models using K-Fold Cross-Validation (GridSearchCV / KFold).
4. **Prediction & Evaluation:** Predict progression target values on the test set for all three models.
5. **Comparison:** Compute evaluation metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Coefficient of Determination (R^2)

Dataset Summary

- **Sample Count:** 442 records, 11 total columns (10 features + 1 target).
- **Features:** age, sex, bmi, bp, s1, s2, s3, s4, s5, s6.
- **Missing Values:** 0 null values across all columns.
- **Target Statistics:** Mean = 152.13, Standard Deviation = 77.09, Min = 25.00, Max = 346.00.

RESULTS & EVALUATION

Metric	Linear Regression (OLS)	Ridge Regression (L2)	Lasso Regression (L1)
MSE	*Evaluated on test set*	*Optimized via Grid Search*	*Optimized via Grid Search*
RMSE	*Evaluated on test set*	*Optimized via Grid Search*	*Optimized via Grid Search*
MAE	*Evaluated on test set*	*Optimized via Grid Search*	*Optimized via Grid Search*
R^2 Score	*Evaluated on test set*	*Optimized via Grid Search*	*Optimized via Grid Search*

Conclusion

Ridge and Lasso regularization successfully constrain model complexity compared to unregularized Linear Regression. Lasso performs feature selection by shrinking less impactful feature coefficients to zero, while Ridge mitigates multicollinearity by shrinking coefficient weights smoothly.